



Health Sentinel

A guardrailed multi-agent nutrition & lifestyle assistant

LangGraph's StateGraph was chosen deliberately over Deep Agents: this system's guardrails are deterministic, code-enforced hard stops — consent gates, rate limits, blocked-category suppression, confidence caps, severity escalation, allergen filtering — and a health-safety system cannot let any of those be an instruction an LLM might skip.

Python 3.12+

LangGraph 1.x

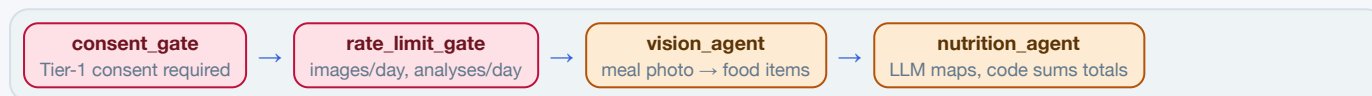
Chroma + SQLite

OpenAI

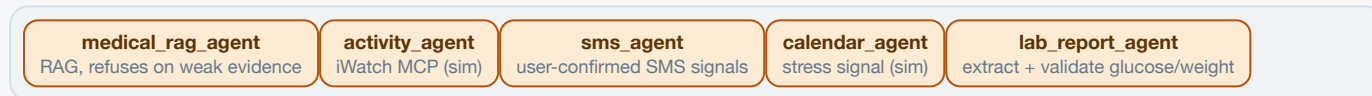
Streamlit

What it is. A LangGraph `StateGraph`: fixed nodes, conditional edges, a real parallel fan-out/fan-in for the MCP + trend agents, a checkpointer, and `interrupt()` for nutritionist HITL review. The deterministic `guardrail_verifier` re-checks raw state directly and is the authoritative pass/fail gate — the LLM `critic_agent` runs only on already-clean runs and can never override a verifier failure.

ENTRY & MEAL ANALYSIS

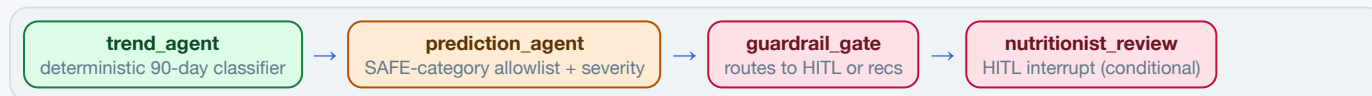


PARALLEL FAN-OUT (CONTEXT AGENTS)

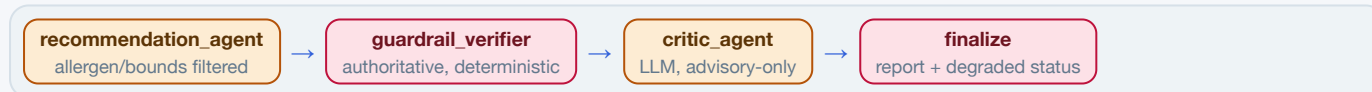


↑ parallel fan-out from nutrition_agent, all five feed into trend_agent below

DETERMINISTIC TREND + PREDICTION + FIRST HITL GATE



RECOMMENDATION, VERIFIER, ADVISORY CRITIC, FINALIZE



Agents & division of labour

- **Eleven nodes, one graph.** vision, nutrition, medical_rag, activity, sms, calendar, lab_report, trend (deterministic), prediction, recommendation, critic — plus consent/rate-limit/verifier gates.
- **Deterministic where possible.** Nutrition totals are a code lookup+sum; trend verdicts are code; the LLM only maps free text or phrases an already-computed verdict.
- **Category allowlist, not a denylist.** prediction_agent keeps only SAFE_CATEGORIES; an unknown category is rejected the same as a blocked one.
- **Per-run spend cap.** Every LLM call is instrumented for tokens/cost; a run hitting the cap short-circuits to a degraded finalize instead of continuing to spend.

RAG & historic trends

- **Hybrid retrieval.** Dense (Chroma/OpenAI embeddings) + lexical overlap fusion, per-user metadata filter, per-document cap, refusal below a 0.28 fused score.
- **Citations tied to real chunk IDs.** Every citation resolves to the exact indexed chunk, not a reconstructed string.
- **SQLite metrics store.** Glucose/weight/sleep readings persist across restarts — the only durable store in the app, because trend reasoning needs history to survive.
- **Deterministic trend classifier.** Rolling mean + threshold-day-fraction math decides stable/improving/worsening; the LLM only phrases the result.

Validation, HITL & safety

- **Deterministic verifier is authoritative.** Re-checks predictions/recommendations/citations directly; a failure always forces nutritionist_review regardless of critic verdict.
- **Two-entry HITL routing.** `review_stage` tracks whether the interrupt came from guardrail_gate (pre-recommendation) or guardrail_verifier (post-verifier) so resume goes to the right node.
- **Checkbox confirmation for SMS.** Every keyword-matched SMS signal is a pre-checked box the user can uncheck — human confirmation as the validation boundary where there's no LLM call to validate.
- **Adversarial eval suite.** Direct-diagnosis and prompt-injection cases require a 100% block rate; allergen-smuggling and confidence-inflation may block or flag.

Resilience & observability

- **Typed fault taxonomy.** Every exception is classified by owning layer + fault class (timeout, schema validation, content filter, provider error, budget exceeded) instead of a free-text string.
- **Every event carries run_id.** Threaded from the graph's thread_id into every audit event so a run can be replayed end to end.
- **Degraded, not silent-success.** A run that recovers from an internal fault finalizes with `run_status="degraded"` and a recovered-fault count, never plain success.
- **Traceability report, not a live UI table.** The audit trail renders to `docs/audit_trace_report.md` and prints to console after every run.

AT A GLANCE

11 agents + gates	4 synthetic RAG profiles	90 day trend window	3 model tiers: cheap / mid / reasoning	\$0.50 hard per-run spend cap	59/59 eval checks passing
-----------------------------	---------------------------------------	-------------------------------	---	--	-------------------------------------